// README · MODULE DOCUMENTATION

Profile *Builder*

Temporal load profile normalisation pipeline. Ingests raw regional electricity withdrawal data, normalises hourly coefficients against the annual integral, and exports a long-format relational lookup table for PostgreSQL injection.

`Python 3.11` `pandas` `numpy`

> **Regional scope:** This documentation covers the **Sicilia** implementation. The identical algorithm — with only schema name and file-path substitutions — applies to **Puglia** and **Calabria**.

FILE	LANGUAGE	OUTPUT
Profile_Builder_Sicilia/	Python	sicilia_consumption_hourly_profile_lookup.csv

REFERENCE YEAR

2024

// STEP 01

Metadata & Calendar Configuration

The script begins by defining all static metadata needed throughout the pipeline: the provincial ISTAT code mapping, the 2024 working/festive day calendar, and the ordered column structure for the wide-format output matrix.

1.1 Provincial ISTAT Code Mapping

A Python dictionary maps each of Sicily's **9 province names** to their official ISTAT `COD_PROV` numeric codes (81–89). These codes are used as foreign keys throughout the SQL pipeline to link profiles to geographic units.

```
# Example mapping
prov_id_map = {
    'Trapani': 81, 'Palermo': 82, 'Messina': 83,
    'Agrigento': 84, 'Catania': 87, ...
}
```

1.2 2024 FR/FS Day Calendar

For each of the 12 months, the number of **working days (FR — Feriale)** and **festive/weekend days (FS — Festivo/Sabato)** in 2024 are recorded. These weights are the denominator for coefficient normalisation: the annual integral is the sum of (hourly kWh × days in that day-type) across all months.

```
days_data = {
    'gen': [19, 12],  # [FR days, FS days] in January
    'feb': [20, 8],
    ...
}
```

1.3 Wide-Format Column Structure

The output wide-format matrix has one column per combination of **month × day-type × hour**: 12 months × 2 day-types × 24 hours = **576 coefficient columns** per row, plus identifier columns (cod_prov, name, sector).

// STEP 02

Data Ingestion & Format Conversion

Raw load profile data is delivered in Excel format. This step performs a batch conversion of all Excel files found in the regional folder to semicolon-delimited CSV, preparing them for pandas ingestion in subsequent steps.

2.1 Batch Excel Discovery

The script uses **glob pattern matching** to discover all `.xlsx` and `.xls` files inside the regional folder (`sicilia/`). Each file represents one province's raw ARERA withdrawal data.

```
excel_files = glob.glob(os.path.join(folder_path, "*.xlsx"))
             + glob.glob(os.path.join(folder_path, "*.xls"))
```

2.2 Excel → CSV Conversion

Each Excel file is read with `pd.read_excel()` and immediately written to a **UTF-8-BOM encoded semicolon-separated CSV**. The file basename (without extension) is used as the province name identifier. Errors are caught and logged without halting the batch.

// STEP 03

Residential Sector Normalisation Kernel

The most complex kernel. Processes the ARERA residential withdrawal profiles per province, separating working-day and festive-day curves, computing the annual energy integral, and dividing each hourly value by that integral to obtain dimensionless coefficients that sum to 1.0 over the weighted year.

3.1 FR / FS Curve Separation

Each CSV contains rows tagged as **Giorno_feriale (FR)**, **SAB (Saturday)**, or **DOM (Sunday)**. Saturday and Sunday rows are grouped and averaged per month/hour to produce a single FS representative curve. FR rows are taken directly.

3.2 **Pivot to Wide Format**

FR and FS data are concatenated and pivoted: the column name encodes `month_daytype_hourN` (e.g. `gen_fr_hour8`), producing a single-row DataFrame with 576 kWh values for that province.

3.3 **Annual Integral & Normalisation**

The **annual total kWh** is computed as:

```
for m in months_order:
    n_fr, n_fs = days_data[m]
    annual_total += sum(FR_hours[m]) * n_fr
    annual_total += sum(FS_hours[m]) * n_fs

coefficients = raw_kwh / annual_total  # ∑ = 1.0
```

Each coefficient represents the **fraction of annual energy** consumed in that specific hour, day-type, and month. Multiplying any building's annual kWh by the matching coefficient yields absolute hourly load.

// STEPS 04 – 05

Industrial, Services & Agriculture Kernels

4.1 **Industrial Kernel — Seasonality**

A single national industrial profile CSV (`Hourly profile_industry.csv`) provides 24-hour load shapes for **January (winter)** and **July (summer)**. The kernel assumes that on festive days, industrial load drops to the **minimum hourly value** of the active season (base-load assumption). The same annual integral logic is applied and coefficients are replicated identically for all 9 provinces.

```
total = sum(
    days_fr * sum(raw_winter) + days_fs * min_winter * 24
    if m in winter_months
    else days_fr * sum(raw_summer) + days_fs * min_summer * 24
    for m in months_order
)
```

5.1 **Services & Agriculture Kernel — Shared Diurnal Profile**

A single hard-coded 24-element diurnal profile is shared between the **services** and **agriculture** sectors. The profile represents a typical commercial/daytime activity curve (peak between 08:00–17:00). On festive days, all hours are set to the minimum value of the FR profile. The same normalisation is applied and the coefficients are identical across all provinces and both sectors.

// STEP 06

Wide → Long Format Reshape

The final step transforms the wide-format matrix (576 columns per row) into a long-format relational table suitable for direct PostgreSQL ingestion as a lookup table.

6.1 **Concatenate All Sectors & Provinces**

All sector DataFrames (residential, industrial, services, agriculture) across all 9 provinces are concatenated into a single wide DataFrame and sorted by `cod_prov` and `sector`.

6.2 **pd.melt() Unpivoting**

`pd.melt()` unpivots the 576 coefficient columns into rows. Each resulting row has one coefficient value. The column name is then parsed to extract **month**, **day-type (FR/FS)**, and **hour** as separate integer/string fields.

```
df_long = wide_df.melt(
    id_vars=['cod_prov', 'name', 'sector'],
    var_name='temp_col', value_name='coefficient'
)
# 'gen_fr_hour8'  →  month=1, fr_fs='FR', hour=8
```

6.3 **Final Schema & CSV Export**

The long-format table is reduced to the 6 relational columns and exported:

OUTPUT FILE: SICILIA_CONSUMPTION_HOURLY_PROFILE_LOOKUP.CSV

`cod_prov`	ISTAT provincial code (integer, 81–89)
`month`	Month number (1–12)
`fr_fs`	Day type: FR (working) or FS (festive/weekend)
`hour`	Hour index (1–24)
`sector`	Energy sector: res / ind / ser / agr
`coefficient`	Dimensionless fraction of annual energy (∑ = 1.0)

📝

Note: The initial draft of this README documentation was generated with the assistance of Claude AI and subsequently reviewed, verified, and edited by the author for accuracy.